Algorithmic Collusion: Where are we and where should we be going?

Ibrahim Abada[*], Joseph E. Harrington, Jr.[†], Xavier Lambin[‡], and Janusz M. Meylahn[§]

August 1, 2025

Abstract

Research on algorithms learning to collude in simulated markets has raised concerns among competition authorities that it could arise in actual markets. Given the diversity of methods and findings, it is an open question whether any particular showing of supracompetitive prices is reason for concern. Anchored in the perspective that the goal of research is to prevent algorithmic collusion in actual markets, we take stock of this body of work and develop a set of criteria for evaluating it and guiding future research.

[*]Grenoble Ecole de Management, 38000 Grenoble, France; ibrahim.abada@grenoble-em.com

[†]Department of Business Economics & Public Policy, The Wharton School, University of Pennsylvania, Philadelphia, PA 19104 USA; harrij@wharton.upenn.edu

[‡]ESSEC Business School, 3 Av. Bernard Hirsch, B.P. 50105, Cergy, 95021, France, and THEMA, xavier.lambin@essec.edu

[§]University of Twente, Drienerlolaan 5, 7522 NB, Enschede, The Netherlands; j.m.meylahn@utwente.nl

1 Introduction

With the arrival of Big Data and advances in machine learning and AI, algorithms are performing a growing role in assisting human agents with their decision-making. One class of decisions that has recently received attention is firms deciding what prices to charge. While there are efficiencies from delegating pricing to an algorithm—for example, it can adjust price more rapidly to high-frequency demand and supply information and do so in a manner that avoids human behavioral biases—there is also the possibility that algorithms may collude and result in supracompetitive prices. Concerns about consumer harm emanating from price-setting algorithms have been expressed by scholars and practitioners, especially at competition authorities. There is an active policy debate regarding how realistic is this concern and whether current competition law and enforcement are currently equipped to handle it.[1] To our knowledge, the phenomenon of algorithms colluding in actual markets has not yet become a reality, but it would be foolish to underestimate what AI is capable of and thus prudent to prepare for this possibility.

Towards that end, two research questions need to be addressed. First, can algorithms actually learn to collude? It has been known since the mid-late 1980s that, in a repeated oligopoly game, it is a stable point (more specifically, a Nash equilibrium) for finite automata to result in supracompetitive prices (i.e., prices in excess of stage game Nash equilibrium prices which is the accepted competitive benchmark in economics).[2] But that finding just tells us that if algorithms learn to collude, then collusion can persist. The more challenging and open question is whether learning algorithms can start from an arbitrary initial point (such as pricing competitively) to evolve to setting supracompetitive prices with some regularity. Second, if algorithms can learn to collude, how do regulators prevent them from colluding and, if collusion occurs, detect and terminate collusion? Executing this two-step research program will require the combined expertise of computer scientists and economists to identify when and how algorithms can learn to collude, and the contribution of legal scholars to design appropriate laws and enforcement policies. This program will require surmounting technological challenges (how do we audit and restrict algorithms?) and legal challenges (is there a violation of existing laws or are new laws needed?).[3]

There is no shortage of research addressing the first question. Table 1 in Appendix B lists more than 30 papers that have come out in the last three years. It is a rich and diverse body of work deploying a variety of learning algorithms to explore the emergence of collusion in a variety of market environments.

The objective of our paper is to develop criteria for evaluating this research and guiding future

[1] See, for example, Mehra (2016), Ezrachi and Stucke (2017), Johnson (2017), Oxera (2017), Deng (2018), Harrington (2018), Kühn and Tadelis (2018), Gal (2019), Schwalbe (2018), Calvano et al. (2020*b*), Beneke and Mackenrodt (2021), Asil and Wollmann (2023), and Ezrachi and Stucke (2023).

[2] See, for example, Rubinstein (1986), Abreu and Rubinstein (1988), and Kalai and Stanford (1988).

[3] In discussing a paper at the 2023 meetings of the Competition & Regulation European Summer School and Conference (CRESSE), one of the authors referred to this research program as the "Frankenstein Program". Like Dr. Victor Frankenstein, who created life only to destroy it, the research program is to create collusion – devise learning algorithms that collude – and then destroy it – figure out how to restrict learning algorithms so they cannot collude.

research. In doing so, we are foremost mindful of the second question: How do we design policies to prevent the harm from algorithms resulting in supracompetitive prices? We believe research addressing the first question should be relevant to addressing the second question.

To appreciate our approach to developing evaluation criteria, consider the neoclassical economic model of firm conduct: profit maximization. It assumes a firm sets prices according to a set of first-order conditions coming from an optimization problem. That model is not viewed as a literal description of firm decision-making but rather is a metaphor which has proven useful for understanding firm conduct. With the profit maximization model, the implicit assumption is that human agents – specifically, a firm's employees - are setting prices. The interpretation is very different when we assume a learning algorithm is setting prices. The learning algorithm is not meant to be a metaphor for some other decision-making process but rather to be a literal description of a firm's learning algorithm. For example, an oligopoly model which has each firm's prices set by Q-learning could be interpreted as a metaphorical model of how a firm's pricing managers dynamically optimize. But that is not the perspective of papers whose objective is to determine if algorithms can collude. Rather, the question they seek to answer is: Can supracompetitive prices emerge when each firm has its prices set by a Q-learning algorithm? If the answer to that question is to inform us about actual market outcomes then one is literally assuming firms are using Q-learning.

Given that perspective, research should be focused on understanding when supracompetitive prices are the outcome of learning algorithms that would plausibly be used by firms. That is the case when (1) a firm's manager is inclined to adopt the learning algorithm and, upon adopting it, (2) a firm's manager does not routinely override the learning algorithm (so the algorithm is determinative of prices), and (3) a firm's manager is not inclined to discard the learning algorithm.[4] In this paper, we explain that those conditions will hold when a learning algorithm is sophisticated, robust, and has a low cost of learning. After describing and motivating these criteria, their value is illustrated by applying them to some recent research on algorithmic collusion.

While we should be focused on learning algorithms that would be used by firms, research involving other learning algorithms can still advance our understanding; we just need to be careful about the conclusions drawn. With learning algorithms that firms are unlikely to use, we are learning about how the algorithm operates but not about actual firms' pricing. In particular, showing a learning algorithm results in supracompetitive prices is not a concern when the learning algorithm is not a plausible candidate to be taken up by a firm. However, if the mechanism by which the learning algorithm generates supracompetitive prices can be argued to be present in more relevant learning algorithms then that knowledge could advance our understanding of how, in practice, algorithms might result in supracompetitive prices.

Section 2 describes the research path that has led us to where we are now on the matter of

[4] Even when it is overridden, a learning algorithm's output could influence the prices that the human operator sets. Thus, it is possible a firm may continue to use a learning algorithm which it regularly overrides. In that case, a model of firm pricing would require both a literal model of the learning algorithm and a metaphorical or literal model of the human operator who overrides it.

algorithms learning to collude. Keeping in mind that the objective is to inform policy for the purpose of avoiding consumer harm from algorithms, Section 3 clarifies the research goal. Section 4 presents an outline of the set of algorithms that are a cause for concern, and Section 5 offers guidance for moving the research on algorithmic collusion forwards. For economists who might not be familiar with the vast array of learning algorithms that have been used to explore collusion, Appendix A briefly describes them. Though this paper is not a survey, Appendix B provides a useful categorization of research on algorithmic collusion.

2 Can Algorithms Learn to Collude?

Since the pioneering work of Waltman and Kaymak (2008), we have known that learning algorithms can result in persistent supracompetitive prices. In that study, firms were modelled as using Q-learning to select quantities in a repeated quantity (Cournot) game with homogeneous goods. The asymptotic behavior of Q-learning often had quantities below stage game Nash equilibrium levels, and thus prices in excess of competitive levels. Using a different learning algorithm for the same market setting, a similar finding of anticompetitive harm from algorithms was shown in Kimbrough and Murphy (2009).

These studies find algorithms resulted in supracompetitive prices, but is this collusion? In the economics literature, a supracompetitive outcome is a product of collusion but is not equivalent to collusion. Collusion does not just refer to high prices, but also how those high prices are sustained. From the theory of repeated games as embodied in the Folk Theorem, collusion in the context of an oligopoly game (and, more broadly, cooperation in the context of any noncooperative game) refers to Nash (or subgame perfect or sequential) equilibria that result in outcomes (such as supracompetitive prices) producing higher payoffs than stage game Nash equilibria.[5] Referring to this body of work as the equilibrium theory of collusion, higher prices and profits (or payoffs) come about through strategies that encompass rewards and punishments.

> Collusion is when firms use history-dependent strategies to sustain supracompetitive outcomes through a reward-punishment scheme which rewards a firm for abiding by the supracompetitive outcome and punishes it for departing from it. If a firm abides by the collusive outcome (e.g., setting a high price) then it is *rewarded* in the future by rival firms continuing to abide by the collusive outcome (e.g., persisting with high prices); while if it departs from the collusive outcome (e.g., setting a low price) then it is *punished* in the future by rival firms acting aggressively to reduce that deviating firm's profits (e.g., lowering prices).[6]

This perspective to collusion goes back to Stigler (1964) and can be found in all standard treatments in economics.[7] There is much to argue for this approach as it identifies three key elements of collusion - coordinating on an outcome, monitoring for compliance with an

[5] References for repeated games and the Folk Theorem include Fudenberg and Tirole (1991) and Mailath and Samuelson (2006).

[6] Harrington (2017), pp. 1-2.

[7] Examples include Friedman (1977), Vives (1999), and Motta (2004).

outcome, and punishing when there is evidence of noncompliance - which map well to actual cartel practices (Harrington (2006)).[8]

From this perspective, the question of whether the observed supracompetitive prices in, say, Waltman and Kaymak (2008) is collusion can then be posed as: Are the high prices the result of strategies (or policy functions) that embody a reward-punishment scheme? That was the driving question in Calvano et al. (2020*a*). They considered Q-learning in the repeated price game with differentiated products.[9] Prices regularly exceeded competitive prices and, most importantly, the adopted policy functions often had the reward-punishment property. To establish this property, they conducted the following experiment. Upon prices settling down to a supracompetitive level, they forced one of the algorithms to set a lower price for one period, thereby changing the history or state, and then had the algorithms respond in an unconstrained manner. The experiment showed that firms' algorithms dropped prices to a low level in response to this departure from high prices, after which prices gradually returned to their previously high levels. Thus, a "deviation" to a low price induced a sequence of periods of low prices and, consequently, low profits ("punishment"). A possible anthropomorphic interpretation is that algorithms, like humans when they collude, learned to expect a price war (and, consequently, a low future payoff) to follow a deviation from collusive prices. The anticipation of this lower future payoff deterred them from undercutting the rival's high price.

Calvano et al. (2020*a*) have shown a learning algorithm can adopt a reward-punishment scheme and the adoption of such a scheme is associated with supracompetitive prices, but is a reward-punishment scheme necessary for producing such prices? While the equilibrium theory of collusion suggests it is necessary, research indisputably shows it is not.[10] Recent studies examine Q-learning in the same market setting as in Calvano et al. (2020*a*) but impose restrictions that, in effect, rule out the adoption of reward-punishment schemes. One approach is to set the discount factor to zero. As a reward-punishment scheme deters an agent from pricing below the supracompetitive price (with its higher current profit) by inducing low future prices (and thus a lower future profit stream), a reward-punishment scheme should not be adopted when the discount factor is zero as then future profits are not valued by the algorithm. Recent simulations with Q-learning have shown that such myopic algorithms can still learn to set supracompetitive prices (Calvano et al. (2020*b*) and Banchio and Mantegazza (2022) for instance). A second approach is to remove the explicit conditioning of Q-values on past actions (or histories). Thus, "history-dependent" policy functions are ruled out by construction and so it is not feasible for such learning algorithms to adopt a reward-punishment scheme. A simple instance of this ap-

[8]The passage from Harrington (2017) p. 2 goes on to say: "Collusion involves an implicit or explicit understanding among firms that ties future rewards and punishments to current behavior and, by doing so, is able to induce compliance with regards to the supracompetitive outcome." Such mutual understanding exists in practice and is critical to establishing a violation of antitrust or competition law, but need not concern us for the purposes of this paper. The matter of understanding leads one into the deep philosophical bog as to whether algorithms understand. The interested reader can find discussions in Harrington (2018) and Asil and Wollmann (2023).

[9]Also see Klein (2021) who considers Q-learning in a repeated setting where two firms make sequential price decisions.

[10]See Abada and Lambin (2023), Banchio and Mantegazza (2022), den Boer, Meylahn and Schinkel (2022), Meylahn and den Boer (2022), Asker, Fershtman and Pakes (2024), and Epivent and Lambin (2024). Waltman and Kaymak (2008) originally established this property for the quantity game with homogeneous goods.

proach is the competitive play between so-called stateless (or memoryless) algorithms, where a value function conditions only on the actions and not the one-period histories. Recent research with stateless Q-learning has shown that, here again, supracompetitive prices can emerge and persist. As an example, Lambin (2024) shows the stateless version of Calvano et al. (2020*b*) and Klein (2021) also reach supracompetitive prices asymptotically.

3 Clarifying the Goal of the Research Program

What is the goal of this research program and how should it be conducted? Should we be concerned with supracompetitive prices only if they are generated by reward-punishment schemes? Or should we care about supracompetitive prices regardless of how they are generated? Even if it is by a learning algorithm that is as simple as being stateless or as myopic as ignoring the impact of its actions on future payoffs? In this section we clarify the research goal, and in the next section we discuss the types of learning algorithms that should be the focus of our attention in achieving that goal.

Our discussion will revolve around the three key elements to an algorithmic approach to collusion: 1) learning algorithms; 2) policy functions; and 3) outcomes. An *outcome* is a vector of actions and perhaps some associated consequences. In the setting of interest, an outcome is a collection of prices (or quantities or some other firm decision variable) and associated consequences include firms' profits and consumer welfare. A "collusive" outcome is, for example, supracompetitive prices (i.e., prices in excess of some competitive benchmark) but could also be prices in excess of some other benchmark such as the prices in place prior to the event of adopting learning algorithms.[11] A *policy function* is a mapping from states (e.g., current market data pertaining to demand and cost as well as recent prices) to the set of feasible actions (such as prices). A "collusive" policy function has the property that, for some states, it produces a collusive outcome such as supracompetitive prices. A reward-punishment scheme is an example of a collusive policy function. Finally, a *learning algorithm* is a mapping from past data (such as past actions and past performance) to the space of policy functions or the space of actions. A "collusive" learning algorithm could be one with the object to produce supracompetitive prices (as opposed to just having the effect of supracompetitive prices). For example, a reinforcement learning algorithm which uses industry profit, not the firm's own profit, as the performance metric is designed to find actions that yield high industry profit, such as monopoly prices.

Harm to consumers comes from supracompetitive prices, not the policy functions that produce those prices nor the learning algorithm that produces the policy function. Though high prices are then ultimately the object of interest, that does not mean the most effective approach to avoiding harm is to focus attention on restricting outcomes. For example, putting an upper bound on the prices that a learning algorithm can set is just a form of price regulation, and it is well recognized that price regulation is often ineffective because of the heavy informational burden;

[11] In private litigation associated with unlawful collusion, the benchmark price (often referred to as the "but for" or counterfactual price) is the price that would have occurred if firms had not colluded (in other words, but for colluding). Counterfactual prices need not be competitive prices. For example, if firms had been engaging in tacit collusion prior to forming a cartel, then counterfactual prices will exceed competitive prices. That is, cartel formation caused the supracompetitive markup to be larger than it would have been in the but-for world.

regulatory agencies rarely know at what level to set price caps or ceilings and, consequently, could create more harm than that which is intended to be alleviated.

A more effective approach to preventing high prices may be to restrict the learning algorithm itself or the policy function that it is permitted to adopt. This is the approach advised in Harrington (2018) and implicit in Calvano et al. (2020*a*). Drawing from the equilibrium theory of collusion, those papers build on the finding that only policy functions with the reward-punishment property can sustain supracompetitive prices. Rather than restrict learning algorithms not to select supracompetitive prices, the approach is to restrict learning algorithms not to adopt reward-punishment policy functions. However, as argued before, reward-punishment policy functions are not required for learning algorithms to result in supracompetitive prices. Thus, a reward-punishment-based approach would fail to close off all avenues through which learning algorithms could harm consumers through supracompetitive prices. For example, requiring policy functions to be myopic or prohibiting explicit conditioning on previous actions using history dependent states would prevent the adoption of reward-punishment policy functions but would not necessarily prevent the setting of supracompetitive prices.

At this point in our discussion, it is important to distinguish between what we will call cognitive collusion and algorithmic collusion. *Cognitive collusion* is when supracompetitive outcomes are produced by a process "involving conscious intellectual activity such as thinking, reasoning, or remembering."[12] Our current understanding is that cognitive collusion requires human agents at competing firms to consciously coordinate their conduct for the purpose of limiting competition. Antitrust or competition law is designed to prohibit cognitive collusion in that high prices are not illegal but rather a particular process by which high prices are achieved (or at least pursued). U.S. jurisprudence established that firms and their employees are in violation of the law when there is an "agreement" to "unreasonably restrain trade," and other jurisdictions have taken the same view.[13] An agreement exists when firms have a "unity of purpose or a common design and understanding, a meeting of minds,"[14] "a conscious commitment to a common scheme designed to achieve an unlawful objective,"[15] or a "concurrence of wills"[16]. All of these terms focus on the same mental state: mutual understanding to restrain competition. This perspective has influenced what is an appropriate remedy, where "remedy" refers to what firms are concretely prohibited from doing. It is meaningless to prohibit a "meeting of minds" because, until we know how to penetrate the human mind, such a prohibition cannot be implemented. Instead, a remedy prohibits overt acts of communication that facilitate achieving an agreement.[17] Cognitive collusion without communication (or evidence of communication) is

[12] Definition of "cognitive". *Merrian-Webster Dictionary* , www.merriam-webster.com/ accessed July 7, 2024.

[13] "By operationalizing the idea of an agreement, antitrust law clarified that the idea of an agreement describes a process that firms engage in, not merely the outcome that they reach. Not every parallel pricing outcome constitutes an agreement because not every such outcome was reached through the process to which the law objects: A negotiation that concludes when the firms convey mutual assurances that the understanding they reached will be carried out." Baker (1993), p. 179.

[14] *Am. Tobacco Co. v. United States*, 328 U.S. 781, 810 66 S. Ct. 1125 (1946)

[15] *Monsanto Co. v. Spray-Rite Serv. Corp.*, 465 U.S. 752,765 104 S. Ct. 1464 (1984)

[16] Judgment of the Court of First Instance of 26 October 2000. *Bayer AG v Commission of the European Communities*

[17] "[F]ew courts have found a conspiracy without some evidence of communication tending to show an agreement." Hovenkamp (2016), p. 243. "[C]ourts have focused primarily on evidence tending to suggest communi-

lawful not because it is not harmful but because there is no remedy.[18]

Turning to algorithmic collusion, how should we define it? At this exploratory stage of the research program, we have decided to put forth a highly inclusive definition of algorithmic collusion: *Algorithmic collusion* is when supracompetitive outcomes are produced by learning algorithms without human design to produce those outcomes. This expansive definition accommodates a range of underlying mechanisms, encompassing high prices supported by reward-punishment schemes, as well as phenomena like "seeming collusion", "collusion by mistake", or "collusive outcomes without collusion" as described by some authors Calvano et al. (2020*a*), den Boer, Meylahn and Schinkel (2022), Abada and Lambin (2023), Colliard, Foucault and Lovo (2022), and Cho and Williams (2024).

Given the objective is to prevent consumer harm, what exactly is to be prohibited when it comes to algorithmic collusion? Though the answer to that question will only come to light as the research program is executed, we can offer some suggested directions. As a start, let us identify where not to look by avoiding false parallels with cognitive collusion. In the case of algorithmic collusion, there is no "agreement" between learning algorithms, no "understanding" nor "mutual understanding", no "consciousness" nor "conscious commitment to a common scheme", no "will" nor "concurrence of wills". Consequently, prohibiting communications that facilitate agreements is an unsuitable approach for avoiding the harm emanating from algorithmic collusion.[19] Rather, remedies will reside in prohibiting certain properties of learning algorithms or policy functions. While human agents also learn (and thus deploy their own brand of learning algorithms) and adopt strategies (in other words, policy functions), restricting learning or strategies is not a route to addressing cognitive collusion because we cannot monitor cognition and thus cannot prohibit certain modes of learning and reasoning. In contrast, it is possible to monitor algorithms and restrict how they learn and the output they generate. Thus, there are remedies which may prove effective with algorithmic collusion but are infeasible with cognitive collusion.[20]

cation has occurred. Although some cases do not involve testimony or documents detailing communication, the courts nevertheless require proof that they conclude justifies an inference that communications took place." Page (2006), p. 447.

[18] As opined by Judge Stephen Breyer: "Courts have noted that the Sherman Act prohibits agreements, and they have almost uniformly held, at least in the pricing area, that such individual pricing decisions (even when each firm rests its own decisions upon its belief that competitors do the same) do not constitute an unlawful agreement under section 1 of the Sherman Act ... [T]hat is not because such pricing is desirable (it is not), but because it is close to impossible to devise a judicially enforceable remedy for "interdependent" pricing. How does one order a firm to set its prices without regard to the likely reactions of its competitors?" *Clamp-All Corp. v. Cast Iron Soil Pipe Inst.*, 851 F.2d 478,484 (1st Cir. 1988)

[19] However, generative AI may be a caveat to that statement should firms use it and allow those programs to communicate with each other. Policies designed for cognitive collusion may then be appropriate for some forms of algorithmic collusion. This thesis is put forward with supporting arguments in Asil and Wollmann (2023).

[20] In principle, firms' human managers could be prohibited from using a reward-punishment scheme with it being enforced by observationally assessing whether firms' prices are consistent with the use of a reward-punishment scheme. The problem is that such an exercise is difficult and fraught with error. Empirical analysis based on firms' prices and market data cannot deliver the requisite level of confidence required by competition law. It must be clear to firms when they are violating the law, and it must be clear to competition authorities and courts when firms are violating the law. Only then will firms know what to do in order to be compliant and be deterred from being noncompliant. In contrast, with algorithmic collusion, we can observe or infer the policy function producing the prices and that opens up other options not available with cognitive collusion.

4 Algorithms of concern

Surveying the vast body of research on algorithmic collusion, it is important to identify when a study's findings possibly warrant concern among policymakers. Algorithms of concern should (1) lead to persistent supracompetitive prices, and (2) be relevant for use in real market environments. Both of these properties will be clarified below. Our goal is to sharpen the boundaries of this set, with the purpose of guiding future research on algorithmic collusion. We conduct our discussion assuming algorithms set prices, but our arguments carry over to other kinds of strategic decisions such as quantities.

4.1 Persistence

An algorithm that sufficiently explores the price space will, by chance, select supracompetitive prices. If an algorithm's initial conditions encompass supracompetitive prices, then a learning process engaging in local search will choose supracompetitive prices during early iterations of learning. These sources of supracompetitive prices do not concern us, for they will fade as algorithms learn and exploration declines slowly. The problem is when, after a reasonable amount of learning and exploration, supracompetitive prices persist.

We define persistence as a property of a particular trajectory of prices. For given initial conditions and stochastic realization of learning (in particular, that associated with random experimentation), supracompetitive prices are persistent if they occur sufficiently frequently after some reasonable amount of learning and exploration. If learning algorithms result in prices converging and do so at supracompetitive levels, then supracompetitive prices are certainly persistent. When prices do not converge, but the distribution of prices does converge, then supracompetitive prices are persistent when the stationary distribution assigns sufficient probability to such prices. A persistence criterion not involving convergence has the fraction of periods with supracompetitive prices being sufficiently high when measured over a sufficiently long sequence of periods. All of these approaches offer a way to conclude that supracompetitive prices are a regular feature of some price series produced by firms' learning algorithms.

Depending on initial conditions and the stochastic realization of learning, some trajectories may result in persistent supracompetitive prices and others not. It is important to distinguish between when persistent supracompetitive prices are an aberration (e.g., requiring special initial conditions) and are a common occurrence. That can be assessed by running the learning algorithms for a variety of initial conditions and realizations of the stochastic learning process. If the percentage of trajectories with persistent supracompetitive prices is sufficiently high, then we conclude these learning algorithms generate a disconcerting frequency of supracompetitive prices.

Recent studies have taken various approaches to establishing the persistence of supracompetitive outcomes. Using Q-learning algorithms, Calvano et al. (2020*a*) find in their simulations that policy functions nearly always converge (using the criterion that they do not change for 100,000 consecutive periods). For convergent policy functions, persistent supracompetitive prices are shown to be very common, with prices typically closer to the monopoly level than the compet-

itive level. Using a large language model, Fish, Gonczarowski and Shorrer (2024) first show for the monopoly case that GPT-4 results in near-optimal pricing within 100 periods, which gives a sense of the time until convergence. For a market with two firms using GPT-4, 21 runs, each consisting of 300 periods, are conducted. Reporting the average prices over periods 251-300, prices always exceed competitive levels, often by a significant amount. The learning algorithm in Cho and Williams (2024) is based on a model-averaging approach, and they find competitive prices are the stable stationary point. However, visual inspection shows that episodes of persistent supracompetitive prices occur for some runs, but the probability of those episodes is proven to go to zero as the probability of experimentation goes to zero. Whether there is a disconcerting frequency of supracompetitive prices requires closer examination in terms of how often they occur for plausible experimentation rates.

4.2 Guiding principles for relevant learning algorithms

A learning algorithm is not relevant when it is unlikely to be adopted by a firm or, when adopted, it is routinely overridden (in which case, the algorithm is not determinative of a firm's prices) or is eventually discarded. The adoption decision is based on an ex ante assessment of how well a learning algorithm will perform, while the overriding and discarding decisions are based on an ex post assessment of how well a learning algorithm has performed.

i) A firm may not adopt a learning algorithm because a human operator believes the output is likely be substantively suboptimal. For example, such an assessment could be reached if the learning algorithm does not use clearly relevant and available data such as competitors' prices.

ii) A firm may discard a learning algorithm because a human operator concludes that performance is consistently low. Low performance can come from a variety of sources. It could be the result of excessive experimentation. It could be due to slow learning, which intrinsically harms performance during the learning phase but also in the long run if it causes the firm to override the learning algorithm's output and thereby interferes with learning. It could also be discarded because, in response to a slight change in the environment, the trained algorithm's performance significantly deteriorates.

iii) A firm may override the output of a learning algorithm because a human operator is able to easily identify a superior action to that which is recommended. Some of the reasons for a pricing algorithm to perform poorly are failing to take into account relevant environmental variables, a slow learning process, or the learning algorithm is ineffective at finding optima.

4.3 Criteria for learning algorithms

When any of the three preceding conditions hold for a learning algorithm, a firm's prices are unlikely to be determined by the output of a pricing policy trained using that learning algorithm. Thus, we want to focus on learning algorithms that are immune to these weaknesses. Towards that end, we propose a set of criteria for a learning algorithm to be *relevant* in the sense that it is likely to be adopted, not to be routinely overridden, and likely to be retained.

The first criterion is that a learning algorithm is *sophisticated* enough for it to be adopted and

not routinely overruled. For example, an algorithm may appear too simplistic to be adopted or may be easily outsmarted and underperform so it is overridden or eventually discarded. At the same time, a sophisticated learning algorithm could design a high-performing pricing algorithm for the current environment but one that is fragile in that it performs very poorly when the environment slightly changes. In that case, the learning algorithm could be adopted but then discarded. This leads us to the second criterion: a relevant learning algorithm should be *robust*. Finally, a learning algorithm may be sophisticated and robust but could take a long time or incur costly errors before finding a high-performing pricing algorithm. As a result, it may be overridden or discarded before it converges. The third criterion is then a *low cost of learning*.

In the remainder of this section, we more fully explain what we mean by sophistication, robustness, and low cost of learning and illustrate these criteria by applying them to some learning algorithms used in recent research.

4.3.1 Sophistication

The fields of algorithmic pricing, game theory, and machine learning have ensured that algorithms available to managers responsible for pricing decisions are sophisticated enough that they generate reasonable profits (at least higher than those humans would generate following simple trading rules). It is rather difficult to properly define or quantify the notion of sophistication, but one can provide some properties that could help frame the notion. Intuitively, a sophisticated algorithm will make reasonable use of the available information it can extract from its environment (past prices and rewards, demand forecasts, etc.) and will strive to build a reasonably realistic modeling of the value function (i.e. the mapping between actions and future discounted payoffs). Naturally, an appreciable degree of sophistication is always inherent to the characteristics of the environment in which the algorithm operates. On the one hand, the algorithm should make use of enough relevant information to make well-informed decisions, but should not use too much information that the learning process is hindered, as taking into account high dimensional information might increase the cost of learning, as discussed below. As an example, a pricing reinforcement learning algorithm which accounts for a very long history of prices when estimating the value function will require a prohibitively large amount of time to learn a reasonable approximation of the function. On the other hand, the degree of complexity of the technology used to build the value function should be appropriate for the environment to avoid both poor efficiency and overfitting effects (the concept of algorithmic technology encompasses various notions including the parametrization of the value function, the frequency of its updating, and the exploration policy). Overfitting refers to a situation where the value function is overly parametrized so that it can infer very well the profits accrued at states that the algorithm saw when learning but will fail to extrapolate properly at new states (out of sample data). In that vein, the basic Q-learning algorithm that uses a simple table approximation of the value function will rapidly lack sophistication when it learns in environments where actions and states are continuous and when the variations of the value function with respect to actions and states are relatively complex.[21]

[21] As an example, Abada and Lambin (2023) suggest that while Q-learning can optimize profits quite well, it can fail to optimize social welfare, a function that is, in their case, quadratic with respect to the actions of the

While it is straightforward that a more sophisticated algorithm will perform better in a single-agent setting (i.e. generate higher profits), the impact of increased sophistication in a multi-agent context is less clear. The work of Cooper, Homem-de Mello and Kleywegt (2015) shows that when competing algorithms account for the actions of competitors when they estimate the demand function before calculating the optimal action to play, they converge to the competitive Nash equilibrium. On the contrary, when they do not account for this information, they use supracompetitive prices asymptotically. In other words, taking into consideration additional relevant information (in this case competitors' actions) and therefore increasing sophistication may render algorithms more competitive. Results with a similar flavor under self-play can be found in the recent literature: In Abada, Lambin and Tchakarov (2024), sophistication is increased by making the approximation of the value function more complex (via neural networks for instance) whereas in Asker, Fershtman and Pakes (2022), sophistication is enhanced by increasing the range of actions for which the approximation of the value function is updated (synchronous learning). In Dawid, Harting and Neugart (2023), sophistication takes the form of a feature called "experience replay", whereby each updating period is trained using not only the most recent observation but a sample from the set of past observations. Interestingly, Abada, Lambin and Tchakarov (2024) also demonstrate that competition involving *asymmetric* algorithms will generate higher profits for the firm using the more sophisticated algorithm, highlighting that there could be an incentive to use reasonably sophisticated algorithms over simple ones also in a multi-agent setting.

Therefore, given the current state of research on algorithmic sophistication in the context of economic collusion, we will follow guiding principle i), by assuming that only algorithms that are reasonably sophisticated with respect to the environment in which they operate will be used by managers.

Example

The simple algorithms of Cooper, Homem-de Mello and Kleywegt (2015) yield supracompetitive profits when they are constrained to ignore market prices; otherwise, prices tend to converge to the Nash equilibrium. Similarly, Lambin (2024) and Abada, Lambin and Tchakarov (2024) show that algorithms constrained in their ability to explore may converge to high prices. The limited sophistication of these algorithms makes them unlikely to be adopted in practice.

4.3.2 Robustness

We define algorithmic robustness as a property of the outcome irrespective of the nature of the outcome itself, i.e., whether prices are supracompetitive or not. More precisely, outcomes should be robust to:

1. Changes in the market environment during the learning process, such as in the demand function. For example, if algorithms learned to price supracompetitively in a static environment where demand follows a certain distribution, then they should still be able to set

algorithm.

supracompetitive prices even for a slight change of this distribution.

2. Reasonable changes in the algorithmic hyperparameters such as the learning rate, exploration rate, and state space specifications.

3. Changes of the algorithms competitors use when these algorithms are relevant and thus reasonably likely to be adopted by a competitor.

For algorithms that lead to persistent supracompetitive prices, an additional robustness consideration is their response to changes in an opponent's algorithm that lead to a change in the nature of their optimal response (for example, from a collusive to a competitive response). A robust algorithm would adapt to such a change for (a significant fraction of) algorithms likely to be adopted by a competitor.

Following guiding principle ii), we hold that only algorithms that are robust will be used by managers. Our definition is inspired by the general notion of robustness in optimization theory (Ben-Tal, El Ghaoui and Nemirovski (2009)), where an objective function (here the profit sustained by supracompetitive pricing) is optimized against a possibly infinite set of constraints (here the conditions of learning, the market environment, the algorithms' hyperparameters, etc.). As in the case of persistence, robustness can be inferred *ex post* by auditing the algorithms once they are put on the market.

Our formulation of robustness is weaker than that suggested in den Boer (2023) which requires algorithms to perform robustly against all alternative algorithms in a class to be specified by the researcher. This is because our proposal does not say anything about the equilibrium of the meta-game of algorithmic selection by managers.[22] Naturally, algorithms satisfying the stricter definition of den Boer (2023) are also a cause for concern. The motivation for a weaker definition is that not all managers are likely to engage in the meta-game of algorithm selection and therefore, assessing robustness with respect to all possible algorithmic alternatives (as in den Boer (2023)) may inhibit policymakers from designing policy that effectively regulates algorithms that may lead to sustained supracompetitive prices in practice.

Example

Eschenbaum, Mellgren and Zahn (2022) show that the high prices learned during training consistently reduce to competitive levels in testing environments. den Boer, Meylahn and Schinkel (2022) and Abada, Lambin and Tchakarov (2024) find that supracompetitive Q-learning algorithms are outperformed by more reasonable alternatives. Similarly, Buchali et al. (2023) and Bichler, Durmann and Oberlechner (2024) suggest that symmetric implementation is a prerequisite for convergence to high prices. Taken together, these findings suggest that these algorithms lack robustness.

[22] The meta-game is a two stage algorithmic selection process. In the first stage, managers chose the algorithm they want to delegate their pricing decision to. In the second stage, the selected algorithms compete in the market. Therefore, the actions of the meta-game are the pricing algorithms available to the managers and the payoffs are the expected rewards obtained in the second stage given continuous use of the selected algorithms.

4.3.3 Cost of learning

As per guiding principle iii), if learning is slow or costly, then the algorithm may never get to deliver supracompetitive profits, as it is likely to be overruled or replaced by human managers. We split the cost of learning into three components:[23]

1. The timescale on which the algorithm learns to price supracompetitively. This depends on the frequency with which decisions can be made and the number of decision episodes required for learning acceptable policies.

2. The profits (or losses) it earns while learning to price supracompetitively.

3. The computational resources required to perform policy updates or the ability to train the algorithm offline.

Only algorithms that have low costs of learning are likely to be used in real markets.

Example

Calvano et al. (2020*b*), Klein (2021) and den Boer, Meylahn and Schinkel (2022) note that the Q-learning algorithms they implement are very slow, often requiring on the order of a million market interactions. The cost of learning makes these algorithms impractical in most real-world applications, at least in their current form.

4.4 Algorithms of concern: Insights from the existing literature

Policymakers are likely to be concerned with the set of algorithms that are both relevant and form part of at least one interaction that leads to persistent supracompetitive prices with other relevant algorithms. Before discussing recent findings of the literature, we begin with a few observations on the use of Generative AI in decision-making.

A note on Generative AI

Large Language Models (LLMs) have recently been studied in the context of algorithmic collusion and have been shown to price supracompetitively. LLMs are highly sophisticated algorithms for next-word-prediction. This fundamentally differs from the task (or objective) of pricing algorithms. The difference is clearest when considering the objectives. A pricing algorithm maximizes profit while a LLM predicts the most likely next token in a string.

Nevertheless, Fish, Gonczarowski and Shorrer (2024) analyze the competitive play between managers, feeding their pricing algorithms with advice from GPT-4. They found that such algorithms persistently set supracompetitive prices under self-play. By changing the 'prompts' algorithms send to the LLM, the authors also suggest that a form of reward-punishment scheme is deployed.

[23] We deliberately split the cost in this way rather than combining time and profits using a discount factor, as the discount factor used by the algorithm may be seen by managers as a (technical) hyperparameter to be tuned that is independent of their time preference. In addition, costs and time may be constrained in different ways, for example, by budget constraints or managers' career concerns.

It is unclear whether LLMs would be able to learn collusive strategies in real-world (stochastic) environments. This differs from traditional pricing algorithms, which are designed to learn by interacting with an environment and can in principle be implemented in real-world settings. It may be possible to augment LLMs with demand learning algorithms or to reprogram them for this purpose (as has been done for time series forecasting by Jin et al. (2023)), but it is unclear whether they would retain their collusive capabilities under such transformations.

Moving on to classical algorithms, some studies have already assessed whether some algorithms may be of concern. Without constituting a systematic literature review, we provide in what follows a summary of some main results found in the extant literature regarding relevant algorithms which generate persistent supracompetitive prices. A short description of the algorithms we will analyze is provided in Appendix A. Furthermore, a summary of the design and findings of some important works analyzing algorithmic collusion is offered in Appendix B.

Competition involving self-play

- **Q-learning.** Recent research has shown that when two or three Q-learning algorithms compete, they can result in persistent supracompetitive outcomes (see Calvano et al. (2020*a*) and Klein (2021) for instance). When the number of algorithms increases, prices tend to decrease and algorithms become more competitive. Evidence regarding the robustness of Q-learning in the context of collusion points towards lack of robustness with respect to changes in the algorithm used by the opponent (den Boer, Meylahn and Schinkel (2022)), and to differences between the training and implementation environment (Eschenbaum, Mellgren and Zahn (2022), Asker, Fershtman and Pakes (2024)). Costs associated with learning supracompetitive outcomes under self-play have been evaluated under the assumption that the discount factor of the manager and the algorithm are identical in den Boer, Meylahn and Schinkel (2022) and they turned out to be very large. In particular, for the rough state and action sets discretization used in Calvano et al. (2020*a*), the authors find that learning to deploy a persistent supracompetitive strategy takes more than 120000 iterations for only two algorithms. If one needs to keep the time of learning below say a month, this would imply that algorithms would need to test more than 4000 prices each day, and that a meaningful estimate of the corresponding demand can be formed for each price, which drastically limits the range of possible applications. This is the result of the tabular nature of Q-learning, which does not exploit structure in the demand or opponents' strategies. Given that demand and opponents' strategies typically exhibit structure and that Q-learning does not exploit this structure, it is questionable if a manager would judge the sophistication of Q-learning to be suitable for deployment in pricing decisions.

- **Myopic and stateless algorithms.** A body of research highlights the possibility of persistent supracompetitive outcomes when myopic or stateless algorithms compete under self-play: Recent examples include Calvano et al. (2020*a*) for myopic Q-learning, Dolgopolov (2024) for stateless Q-learning, and Cho and Williams (2024) for myopic averaging

algorithms, to cite but a few.[24] By construction, these algorithms shut down the adoption of reward-punishment schemes but can still generate supracompetitive outcomes. They are, however, unlikely to fall into the category of algorithms of concern as they obviously lack sophistication and are also probably not robust as highlighted in Abada, Lambin and Tchakarov (2024) where it is shown that outcomes of stateless Q-learning are no longer supracompetitive if there is sufficient exploration.

- **Advanced versions of Q-learning.** Enhancements of Q-learning to estimate both the value function and the optimal policy or to exploit the sophistication of neural networks exist. In Deep Q-learning, the table approximating the value function is replaced by a neural network that could capture some non-linearities via the so-called activation functions, while keeping the discretization of the set of actions. Furthermore, Actor-Critic Q-learning (ACQ) leverages the dynamics of Q-learning to estimate both the value function (the critic) and the optimal policy function (the actor). Using numerical simulations, Hettich (2021) finds existence of sets of Deep Q-learning algorithms that lead to persistent supracompetitive prices under self-play. Possnig (2023) theoretically analyzes the dynamics of Actor-Critic Q-learning under self-play to derive conditions on the structure of the payoffs (of the game) that lead to persistent supracompetitive prices. While these algorithms are more sophisticated than basic Q-learning, we are not aware of research investigating the extent to which they are persistent or robust.

- **Other applications of Neural Networks.** Abada, Lambin and Tchakarov (2024) have shown that a class of reasonably sophisticated neural networks algorithms, such as Actor-Critic (AC) algorithms, do not produce persistent supracompetitive outcomes under self-play in a standard Cournot game. Enhancing the modeling of the actor and the critic in ACQ, these algorithms employ two interconnected neural networks to estimate the value function (the critic) and the optimal pricing policy (the actor). In particular, this makes them belong to the so-called set of policy optimization algorithms. Their main advantage with respect to Q-learning (and its sophisticated variants) is that they operate in continuous state and action spaces and their exploration policy is naturally integrated into the current estimate of the optimal policy.[25] The result that such algorithms do not learn to price supracompetitively might seem surprising, at least if one is to rationalize the high algorithmic prices with a form of cooperation: Increased sophistication should, if anything, facilitate the discovery of cooperative strategies. In parallel, AC algorithms seem to show some robustness (at least with respect to some changes in the competing algorithms as reported in Abada, Lambin and Tchakarov (2024)).

- **Other policy optimization algorithms.** Without necessarily resorting to complex neural networks, the existing literature seems to suggest that simple policy optimization based algorithms interact in a competitive way under self-play. The case of the *Rein-*

[24] Diverging from the traditional literature, Cho and Williams (2024) do not analyze reinforcement learning algorithms. Instead, they study algorithms that optimize their hyperparameters by averaging their estimation of the reaction function of competitors with no discounting.

[25] This is a stark difference from Q-learning in which the action and state spaces are discretized, and the exploration process is fixed *ex ante* by the user.

force algorithm (Williams (1992)) is treated in Abada, Lambin and Tchakarov (2024) and the case of the *Exp3* algorithm (Lattimore and Szepesvári, 2020) is treated in den Boer, Meylahn and Schinkel (2022) and Bichler, Durmann and Oberlechner (2024). The Reinforce algorithm differs from Q-learning by employing gradient descent methods to directly optimize the policy function without estimating the value function. On the other hand, the Exp3 algorithm is a method to solve the multi-armed bandit problem with a proper policy optimization approach (here again without estimating the value function). Unlike Q-learning, Exp3 operates without a state representation. Furthermore, simple competing Natural Policy Gradient methods have been shown to converge to the Nash equilibrium in Wang (2022). This implies the role of policy optimization methods for producing pricing algorithms that persistently generate supracompetitive prices have not been brought to light yet.

- **Provably convergent algorithms.** Algorithms with theoretical performance guarantees are most likely to be robust against a variety of alternative algorithms and have the appropriate level of sophistication for use by managers. Proving that multiagent learning algorithms converge in Markov games is difficult, especially in the decentralized setting in which the information exchange between the algorithms is limited[26]. However, recent advances in the theory of decentralized multiagent reinforcement learning have led to algorithms with theoretical performance guarantees (see Yang and Wang (2020); Zhang, Yang and Başar (2021) for recent surveys). One of these, namely, Decentralized Q-learning proposed in Arslan and Yüksel (2016), has recently been analyzed in the context of algorithmic collusion in Meylahn and Janssen (2022); Meylahn (2023*a*,*b*), where the authors quantify the likelihood that Decentralized Q-learning prices supracompetitively under self-play. This algorithm utilizes a two timescale learning approach in which the policy is kept fixed during batches of interactions. After each batch, the interaction samples obtained during the batch are used to update the policy. In relation to the framework proposed here, the use of two timescale learning, suggests that the algorithm is more *sophisticated* than standard Q-learning. The results of Meylahn and Janssen (2022); Meylahn (2023*b*) address the robustness of Decentralized Q-learning by showing that the outcomes are robust to changes in initial conditions, changes in the hyperparameters, and changes in the market environment. However, Decentralized Q-learning still suffers from requiring prohibitively many episodes to learn to price supracompetitively[27]. For this reason, it most likely is not relevant (although Barfuss and Meylahn (2023) show numerically that slight modifications of the algorithm, which speed up the learning, increase the likelihood of learning to price supracompetitively significantly)[28].

Competition involving different (asymmetric) algorithms

Most of the extant literature examined algorithmic collusion under self-play because of ease of implementation and theoretical analysis. This is why competition involving different algorithms

[26]This is the setting appropriate for algorithmic collusion given the restriction on communication.

[27]It takes on the order of 10^6 time steps to be guaranteed to learn an equilibrium strategy in the case of two prices and a memory of one period.

[28]For a large part of the parameter space, the likelihood of learning the collusive equilibrium is close to one.

is still scarce in the field of algorithmic collusion. The few results we are aware of in that field are the following:

- **Q-learning versus reasonable alternatives.** den Boer, Meylahn and Schinkel (2022) demonstrate that Q-learning performs poorly against the simple *Exp3* algorithm, and that the resulting long-run outcome is no longer supracompetitive for a sufficiently high discount rate. Bichler, Durmann and Oberlechner (2024) investigate Q-learning's performance against a variety of multi-armed bandit algorithms and find that they outperform Q-learning and lead to competitive price among themselves. A similar result can be found in Abada, Lambin and Tchakarov (2024) with Actor-Critic neural networks and with the Reinforce algorithm. This implies that, to the best of our knowledge, at least in the settings studied in the literature so far, sets that include Q-learning and non-Q-learning algorithms do not seem to constitute persistent sets of supracompetitive algorithms.

Designed-to-collude algorithms.
Learning to set supracompetitive prices can be achieved using classical techniques from stochastic approximation algorithms, as shown in Meylahn and den Boer (2022). More advanced techniques have been employed to learn to price supracompetitively by Loots and den Boer (2023) and Loots (2024). Similarly, multi-armed bandit algorithms have been shown to collude in assortment games by Aouad and den Boer (2021). What unifies these algorithms is that they have been designed with the goal of learning to price supracompetitively. This is evident from the objectives being optimized by the algorithms. They therefore do not satisfy the condition of (autonomous) algorithmic collusion put forth here, as discussed in Section 3. Nevertheless, algorithms that are designed to collude may present a threat to social welfare. The tools available to policymakers for regulating their use, however, may differ significantly from those necessary for regulating algorithmic collusion as defined here. In addition, as research progresses on the design elements of algorithms that promote collusion, the distinction between algorithms designed to collude and those that learn to collude autonomously is likely to become blurred.

5 Moving forward

In this paper, we present a methodology to evaluate whether algorithms learning to collude should be a concern for policymakers in terms of their potential to harm consumers. So as to avoid stunting research by prescribing criteria that become discordant with the evolving understanding of algorithmic collusion, the proposed methodology is intentionally broad and flexible and should be shaped as the nascent field of algorithmic collusion grows.

The contribution of this paper targets two audiences. Firstly, this methodology serves as guidance for researchers in the field of algorithmic collusion. It provides a perspective that facilitates identifying relevant research questions and assessing the practical importance of their findings. In this vein, researchers should evaluate the algorithms they examine in terms of their robustness and sophistication and quantify the cost of learning to collude. Additionally, researchers should scrutinize their findings of supracompetitive prices from the perspective of persistence, by quantifying the frequency with which high prices occur.

Secondly, we aim to provide guidance to regulators concerned with the possibility of algorithms learning to charge supracompetitive prices. We intentionally do not prescribe specific targets regarding the frequency of high prices, the mechanisms behind them, or the likelihood of certain algorithms being used together. Indeed, we recognize that determining the level of concern and associated risk will ultimately involve weighing the harm from possible high prices against possible efficiency losses from restricting algorithms. Still, our methodology serves as a useful-framework by establishing some fundamental principles for determining relevant harm which is suggestive of the tools that may be necessary for regulating algorithms.

References

Abada, Ibrahim, and Xavier Lambin. 2023. "Artificial Intelligence: Can Seemingly Collusive Outcomes be Avoided?" *Management Science*, 69(9): 5042–5065.

Abada, Ibrahim, Xavier Lambin, and Nikolay Tchakarov. 2024. "Collusion by mistake: Does algorithmic sophistication drive supra-competitive profits?" *European Journal of Operational Research*, 318(3): 927–953.

Abreu, Dilip, and Ariel Rubinstein. 1988. "The Structure of Nash Equilibrium in Repeated Games with Finite Automata." *Econometrica*, 1259–1281.

Achiam, Josh, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. "Gpt-4 technical report." *arXiv preprint arXiv:2303.08774*.

Aouad, Ali, and Arnoud V. den Boer. 2021. "Algorithmic Collusion in Assortment Games." *Available at SSRN 3930364*.

Arslan, Gürdal, and Serdar Yüksel. 2016. "Decentralized Q-learning for Stochastic Teams and Games." *IEEE Transactions on Automatic Control*, 62(4): 1545–1558.

Arunachaleswaran, Eshwar Ram, Natalie Collina, Sampath Kannan, Aaron Roth, and Juba Ziani. 2024. "Algorithmic Collusion without Threats." *arXiv preprint arXiv:2409.03956*.

Asil, Aslihan, and Thomas Wollmann. 2023. "Can Machines Commit Crimes Under US Antitrust Laws?" *Available at SSRN 4527411*.

Asker, John, Chaim Fershtman, and Ariel Pakes. 2022. "Artificial Intelligence, Algorithm Design and Pricing." *AER, Papers and Proceedings*, 112: 452–456.

Asker, John, Chaim Fershtman, and Ariel Pakes. 2024. "The Impact of Artificial Intelligence Design on Pricing." *Journal of Economics & Management Strategy*, 33(2): 276–304.

Baker, Jonathan B. 1993. "Two Sherman Act Section 1 Dilemmas: Parallel Pricing, the Oligopoly Problem, and Contemporary Economic Theory." *Antitrust Bulletin*, 38(1): 143–219.

Banchio, Martino, and Giacomo Mantegazza. 2022. "Adaptive Algorithms and Collusion via Coupling." *arXiv preprint arXiv:2202.05946*.

Barfuss, Wolfram, and Janusz M. Meylahn. 2023. "Intrinsic Fluctuations of Reinforcement Learning Promote Cooperation." *Scientific Reports*, 13(1): 1309.

Beneke, Francisco, and Mark-Oliver Mackenrodt. 2021. "Remedies for Algorithmic Tacit Collusion." *Journal of Antitrust Enforcement*, 9(1): 152–176.

Ben-Tal, Aharon, Laurent El Ghaoui, and Arkadi Nemirovski. 2009. *Robust Optimization.* Vol. 28, Princeton University Press.

Bichler, Martin, Julius Durmann, and Matthias Oberlechner. 2024. "Online Optimization Algorithms in Repeated Price Competition: Equilibrium Learning and Algorithmic Collusion." *arXiv preprint arXiv:2412.15707*.

Buchali, Katrin, Jens Grüb, Matthias Muijs, and Ulrich Schwalbe. 2023. "Strategic choice of price-setting algorithms." Hohenheim Discussion Papers in Business, Economics and Social Sciences.

Calvano, Emilio, Giacomo Calzolari, Vincenzo Denicoló, and Sergio Pastorello. 2020*a*. "Artificial Intelligence, Algorithmic Pricing, and Collusion." *American Economic Review*, 110(10): 3267–97.

Calvano, Emilio, Giacomo Calzolari, Vincenzo Denicoló, Joseph E. Jr. Harrington, and Sergio Pastorello. 2020*b*. "Protecting Consumers from Collusive Prices Due to AI." *Science*, 370(6520): 1040–1042.

Cartea, Álvaro, Patrick Chang, and José Penalva. 2022. "Algorithmic Collusion in Electronic Markets: The Impact of Tick Size." *Available at SSRN 4105954*.

Cartea, Álvaro, Patrick Chang, José Penalva, and Harrison Waldon. 2022. "The Algorithmic Learning Equations: Evolving Strategies in Dynamic Games." *Available at SSRN 4175239*.

Cartea, Álvaro, Patrick Chang, José Penalva, and Harrison Waldon. 2023. "Algorithms can Learn to Collude: A Folk Theorem from Learning with Bounded Rationality." *Available at SSRN 4293831*.

Cho, In-Koo, and Noah Williams. 2024. "Collusive Outcomes Without Collusion." *arXiv preprint arXiv:2403.07177*.

Colliard, Jean-Edouard, Thierry Foucault, and Stefano Lovo. 2022. "Algorithmic Pricing and Liquidity in Securities Markets." *HEC Paris Research Paper No. FIN-2022-1459*.

Conjeaud, Ivan. 2023. "Algorithmic Collusion under Competitive Design." *arXiv preprint arXiv:2312.02644*.

Cooper, William L., Tito Homem-de Mello, and Anton J. Kleywegt. 2015. "Learning and Pricing with Models that Do Not Explicitly Incorporate Competition." *Operations Research*, 63(1): 86–103.

Dawid, Herbert, Philipp Harting, and Michael Neugart. 2023. "Implications of Algorithmic Wage Setting on Online Labor Platforms: a Simulation-Based Analysis." *Available at SSRN 4405558*.

den Boer, Arnoud V. 2023. "A (Mathematical) Definition of Algorithmic Collusion." *Available at SSRN 4636488*.

den Boer, Arnoud V., Janusz M. Meylahn, and Maarten Pieter Schinkel. 2022. "Artificial Collusion: Examining Supracompetitive Pricing by Q-Learning Algorithms." *Amsterdam Law School Research Paper No. 2022-25, Amsterdam Center for Law & Economics Working Paper No. 2022-06*.

Deng, Ai. 2018. "What Do We Know About Algorithmic Tacit Collusion." *Antitrust*, 33: 88.

Deng, Shidi, Maximilian Schiffer, and Martin Bichler. 2024. "Algorithmic Collusion in Dynamic Pricing with Deep Reinforcement Learning." *arXiv preprint arXiv:2406.02437*.

Dolgopolov, Arthur. 2024. "Reinforcement Learning in a Prisoner's Dilemma." *Games and Economic Behavior*, 144: 84–103.

Epivent, Andréa, and Xavier Lambin. 2024. "On Algorithmic Collusion and Reward–punishment Schemes." *Economics Letters*, 237: 111661.

Eschenbaum, Nicolas, Filip Mellgren, and Philipp Zahn. 2022. "Robust Algorithmic Collusion." *arXiv preprint arXiv:2201.00345*.

Ezrachi, Ariel, and Maurice E. Stucke. 2017. "Two Artificial Neural Networks Meet in an Online Hub and Change the Future (Of Competition, Market Dynamics and Society)." *Oxford Legal Studies Research Paper No. 24/2017; University of Tennessee Legal Studies Research Paper No. 323.*

Ezrachi, Ariel, and Maurice E. Stucke. 2023. "Algorithmic Tacit Collusion." In *Research Handbook on Cartels.* 187–212. Edward Elgar Publishing.

Fish, Sara, Yannai A. Gonczarowski, and Ran I. Shorrer. 2024. "Algorithmic Collusion by Large Language Models." *arXiv preprint arXiv:2404.00806.*

Friedman, James W. 1977. *Oligopoly and the Theory of Games.* North-Holland.

Fudenberg, Drew, and Jean Tirole. 1991. *Game Theory.* MIT press.

Gal, Michal S. 2019. "Algorithms as Illegal Agreements." *Berkeley Technology Law Journal*, 34(1): 67–118.

Harrington, Joseph E., Jr. 2006. "How do Cartels Operate?" *Foundations and Trends in Microeconomics*, 2(1): 1–105.

Harrington, Joseph E., Jr. 2017. *The Theory of Collusion and Competition Policy.* MIT Press.

Harrington, Joseph E., Jr. 2018. "Developing Competition Law for Collusion by Autonomous Artificial Agents." *Journal of Competition Law and Economics*, 14(3): 331–363.

Hettich, Matthias. 2021. "Algorithmic Collusion: Insights from Deep Learning." *Available at SSRN 3785966.*

Hovenkamp, Herbert. 2016. "Federal antitrust policy: the law of competition and its practice." *5th edition, West Academic Publishing.*

Jin, Ming, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. 2023. "Time-llm: Time Series Forecasting by Reprogramming Large Language Models." *arXiv preprint arXiv:2310.01728.*

Johnson, Paul A. 2017. "Should We Be concerned that Data and Algorithms Will Soften Competition?" *CPI Antitrust Chronicle*, 2: 10–15.

Kalai, Ehud, and William Stanford. 1988. "Finite Rationality and Interpersonal Complexity in Repeated Games." *Econometrica*, 397–410.

Kimbrough, Steven O, and Frederic H Murphy. 2009. "Learning to Collude Tacitly on Production Levels by Oligopolistic Agents." *Computational Economics*, 33: 47–78.

Klein, Timo. 2021. "Autonomous Algorithmic Collusion: Q-learning under Sequential Pricing." *RAND Journal of Economics*, 52(3): 538–558.

Kühn, Kai-Uwe, and Steve Tadelis. 2018. "The Economics of Algorithmic Pricing: Is Collusion Really Inevitable?" working paper.

Lambin, Xavier. 2024. "Less Than Meets the Eye: Simultaneous Experiments as a Source of Algorithmic Seeming Collusion." *Available at SSRN 4498926.*

Lattimore, Tor, and Csaba Szepesvári. 2020. *Bandit Algorithms.* Cambridge University Press.

Loots, Thomas. 2024. "Algorithmic Collusion in a Price Oligopoly." PhD diss. University of Amsterdam.

Loots, Thomas, and Arnoud V. den Boer. 2023. "Data-driven Collusion and Competition in a Pricing Duopoly with Multinomial Logit Demand." *Production and Operations Management*, 32(4): 1169–1186.

Macrì, Andrea, and Fabrizio Lillo. 2024. "Deviations from the Nash equilibrium in a two-player optimal execution game with reinforcement learning." *Available at SSRN 5102113.*

Mailath, George J., and Larry Samuelson. 2006. *Repeated Games and Reputations: Long-run Relationships.* Oxford university press.

Mehra, Salil K. 2016. "Antitrust and the Robo-Seller: Competition in the Time of Algorithms." *Minnesota Law Review*, 100: 1323–1375.

Meylahn, Janusz M. 2023*a*. "Does an Intermediate Price Facilitate Algorithmic Collusion?" *Available at SSRN 4594415.*

Meylahn, Janusz M. 2023*b*. "Quantifying the Likelihood of Learning Collusive Strategy Equilibria." *Available at SSRN 4589989.*

Meylahn, Janusz M., and Arnoud V. den Boer. 2022. "Learning to Collude in a Pricing Duopoly." *Manufacturing & Service Operations Management*, 24(5): 2577–2594.

Meylahn, Janusz M, and Lars Janssen. 2022. "Limiting Dynamics for Q-learning with Memory One in Symmetric Two-player, Two-action Games." *Complexity*, 2022(4830491): 1–20.

Motta, Massimo. 2004. *Competition Policy: Theory and Practice.* Cambridge University Press.

Page, William H. 2006. "Communication and Concerted Action." *Loyola University Law Journal*, 38: 405.

Possnig, Clemens. 2023. *Reinforcement Learning and Collusion.* Department of Economics, University of Waterloo.

Rubinstein, Ariel. 1986. "Finite Automata Play the Repeated Prisoner's Dilemma." *Journal of Economic Theory*, 39(1): 83–96.

Sadoune, Igor, Marcelin Joanis, and Andrea Lodi. 2024. "Algorithmic Collusion and the Minimum Price Markov Game." *arXiv preprint arXiv:2407.03521.*

Sanchez-Cartas, J Manuel, and Evangelos Katsamakas. 2022. "Artificial Intelligence, Algorithmic Competition and Market Structures." *IEEE Access*, 10: 10575–10584.

Schaefer, Maximilian. 2022. "On the Emergence of Cooperation in the Repeated Prisoner's Dilemma." *arXiv preprint arXiv:2211.15331.*

Schlechtinger, Michael. 2024. "Investigating, Predicting, and Mitigating Collusive Behavior in Deep Reinforcement Learning-based Pricing AIs."

Schwalbe, Ulrich. 2018. "Algorithms, Machine Learning, and Collusion." *Journal of Competition Law & Economics*, 14(4): 568–607.

Stigler, George J. 1964. "A Theory of Oligopoly." *Journal of Political Economy*, 72(1): 44–61.

Vives, Xavier. 1999. *Oligopoly Pricing: Old Ideas and New Tools.* MIT Press.

Waltman, Ludo, and Uzay Kaymak. 2008. "Q-learning Agents in a Cournot Oligopoly Model." *Journal of Economic Dynamics and Control*, 32(10): 3275–3293.

Wang, Qiaochu, Yan Huang, Param Vir Singh, and Kannan Srinivasan. 2023. "Algorithms, artificial intelligence and simple rule based pricing." *Available at SSRN 4144905.*

Wang, Yanwen. 2022. "Will Algorithms Learn to Collude? Insights from the Natural Policy Gradient Method." *Distinguished Majors Program, Department of Economics, University of Virginia.*

Wang, Z., V. Bapst, N. Heess, V. Mnih, R. Munos, K. Kavukcuoglu, and N. de Freitas. 2016. "Sample Efficient Actor-critic with Experience Replay." *arXiv preprint arXiv:1611.01224.*

Werner, Tobias. 2021. "Algorithmic and Human Collusion." *Available at SSRN 3960738.*

Williams, Ronald J. 1992. "Simple Statistical Gradient-following Algorithms for Connectionist Reinforcement Learning." *Machine Learning*, 8: 229–256.

Xu, Zhang, and Wei Zhao. 2024. "On Mechanism Underlying Algorithmic Collusion." *arXiv preprint arXiv:2409.01147.*

Yang, Yaodong, and Jun Wang. 2020. "An Overview of Multi-agent Reinforcement Learning from Game Theoretical Perspective." *arXiv preprint arXiv:2011.00583.*

Zhang, Kaiqing, Zhuoran Yang, and Tamer Başar. 2021. "Multi-agent Reinforcement Learning: A Selective Overview of Theories and Algorithms." *Handbook of reinforcement learning and control*, 321–384.

Zhang, Weipeng. 2023. "Pricing via Artificial Intelligence: The Impact of Neural Network Architecture on Algorithmic Collusion." *Available at SSRN 4489688.*

A A description of the algorithms considered in this work

In this section, we outline the functioning of the algorithms considered in this article. Except for Large Language Modeling algorithms, such as GPT-4, which are trained via supervised learning on an extensive database extracted from the web and employ techniques of Natural Language Processing (Achiam et al. (2023)), all other algorithms discussed are based on Reinforcement Learning (RL). Unlike supervised learning, RL algorithms do not necessarily require a pre-existing dataset for training, which constitutes one of their main advantage, as they do not necessitate a model of their environment. Of course, while learning can be accelerated by guiding these algorithms with some information on desired behaviors such as undercutting rivals, aligning strategies with specific rules to abide by regulation, etc., this assistance is not necessary. Indeed, RL algorithms can learn autonomously from scratch: initially, they do not need to know anything about the system in which they operate, the size of the market, the demand function, the number of competitors, etc., as some of this information can be implicitly inferred during training.

In a nutshell, an RL algorithm observes a *state* of the system and tries to take the *action* that maximizes its *objective*. To ease the exposition in what follows, we focus on pricing algorithms (the action is the price) which strive to maximize profit (the objective) in a given market (the environment). The objective of RL is to find the mapping between a (state, price) couple, which we denoted here by (s, p), and the optimal long-term discounted profit accrued if action p is played when the algorithm observes state s, which we denote by $\Pi(s, p)$. In analogy with Dynamic Programming, this function is sometimes referred to as the *value function*:

$$\Pi(s,p) = \pi(s,p) + \delta\mathbb{E}\Big(\mathrm{Max}_{p'\in\mathcal{P}}\Pi(s'(s,p),p')\Big), \tag{1}$$

where $\pi(s, p)$ is the immediate profit accrued from the market, $\mathcal{P}$ is the set of feasible prices, δ is the discount rate, and $s'(s, p)$ is the random state to which the environment transitions following an unknown probability distribution. All the algorithms we are about to present are theoretically elaborated assuming a static environment although, in practice, they are used in all kinds of environments.

An RL algorithm is updated iteratively by considering past actions and their corresponding rewards. In the first iterations, actions are taken randomly, as the algorithm did not build enough 'knowledge' about the environment. This is the exploration phase. If the algorithm's developer has a synthetic model of the environment, this process can be undertaken offline (not in the market), limiting thereby the cost of exploration due to inefficient actions.

Since some of the RL algorithms we examine utilize neural networks, we will begin with a broad overview of these.

A.1 Neural networks

Schematically, the main objective of a neural network (NN) is to find a mapping between an input vector $x = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$ and an output vector $y = (y_1, y_2, \ldots, y_q) \in \mathbb{R}^q$ (or $[0, 1]^q$

for classification algorithms). To do so, a neural network stacks a number $L \in \mathbb{N}$ of layers with $L \geq 2$. Each layer l contains n_l units (or neurons) which receive a 'signal' from previous layers in a recursive way, as we explain below. Each layer l is also defined by some weights $w^l_{i,j} \in \mathbb{R}$ with $1 \leq i \leq n_{l+1}$, $1 \leq j \leq n_l + 1$, and $1 \leq l \leq L-1$. In its most basic functioning (a fully connected network), the algorithm calculates intermediary variables z_l recursively as follows:

$$z^1 = \tilde{x} \tag{2}$$

$$z^{l+1}_i = g\Big(\sum_{j=1}^{n_l+1} w^l_{i,j}\tilde{z}^l_j\Big),\ 1 \leq i \leq n_{l+1} \text{ and } 1 \leq l \leq L-1, \tag{3}$$

where, for a given vector $a = (a_1, a_2, \ldots, a_t) \in \mathbb{R}^t$, $\tilde{a}$ denotes the vector $(a_1, a_2, \ldots, a_t, 1) \in \mathbb{R}^{t+1}$, $n_1 = n$, and $n_L = q$. Function g is the so-called activation function, which allows capturing non-linearities in the model. Standard activation functions include the logistic, the hyperbolic tangent, or the Rectified Unit Linear functions. The activation function is assumed to be the same across layers in this description, but this is not restrictive, as one can make it depend on indices l and i. The output variable z^L corresponding to the last layer is the approximation of output variable y. Therefore, the algorithm will strive to make these variables as close as possible, by tuning its weights $w^l_{i,j}$.

A neural network is a supervised learning algorithm. Given o input observations $\mathbf{x} = (x^1, x^2, \ldots, x^o) \in \mathbb{R}^{o\times n}$ and their corresponding outputs $\mathbf{y} = (y^1, y^2, \ldots, y^o) \in \mathbb{R}^{o\times q}$, we define a loss function (or an error of prediction function) $\mathcal{L}\Big(\mathbf{x}, \mathbf{y}, \mathbf{w}\Big)$ where $\mathbf{w}$ is the concatenation of weights $w^l_{i,j}$. The purpose of the algorithm is to seek the weights $\mathbf{w}$ that minimize the loss function, a function which, depending on the choice of the activation functions, can be made convex. A standard procedure to achieve this goal is to use the gradient descent algorithm or its stochastic alternative when the dataset is large. The calculation of the gradient is usually achieved via backward propagation, where one goes through the network from the last layer to the first. Following the gradient descent logic, the neural network will iteratively update its weights until convergence.

A.2 Q-learning

Q-learning (QL) is an RL algorithm in which the sets of states and actions are discretized. The algorithm is iterative, with time also discretized into steps $n \in \mathbb{N}$. A state can encompass any relevant and observable market information, such as past prices (including those of competitors), past rewards, consumption, etc. For instance, if two agents operate in the market and prices are discretized into $p_1,\ p_2, \ldots, p_m$ values, then a state observed at iteration n can be a couple (p^1_{n-1}, p^2_{n-1}) where p^i_k denotes the action taken by agent i at time (or iteration) k. Therefore, assuming that both agents use the same discretization of prices, the set of states $\mathcal{S}$ is equal to $\mathcal{P}^2$ in this case, where $\mathcal{P}$ denotes set $\{p_1, p_2, \ldots, p_m\}$. Of course, this representation extends naturally to a longer memory and various action sets and market agents.

QL strives to estimate the long-term discounted profit $\Pi(s, p)$ in the so-called Q-matrix: $Q_{i,j}$ with $1 \leq i \leq |\mathcal{S}|$, $1 \leq j \leq m$, which can sometimes also be denoted by $Q(s_i, p_j)$, is the estimation of the value function when state s_i occurs and price p_j is set. Following the logic of RL, at iteration n, the algorithm observes the state of the system s_n and chooses the price it sets p_n

according to a probability distribution $\theta_n(\cdot)$. This distribution tunes the pace at which the algorithm explores. For instance, in the early stages of learning, the distribution is broad to ensure the algorithm explores all possible actions and learns about their corresponding rewards. After some iterations, the algorithm can then be used to play its optimal action increasingly often with the purpose to maximize the long-term profit. Because exploration is costly inasmuch as the algorithm might take inefficient actions, in concrete applications of learning, distribution $\theta_n(\cdot)$ is rapidly changed to focus on the profit-maximizing actions. The simplest exploration policy is the so-called epsilon-greedy: if we denote by Q_n the estimation of the Q-matrix at iteration n, then if the algorithm sees state s_n, the optimal decision, $p^* = \text{argmax}_{p\in\mathcal{P}} Q_n(s_n, p)$, is chosen at probability $1 - \epsilon(n)$:

$$\theta_n(p^*) = 1 - \epsilon(n), \tag{4}$$

and the other prices $p \neq p^*$ are played with equal probability $\frac{\epsilon(n)}{m-1}$:

$$\theta_n(p) = \frac{1}{m-1}\epsilon(n) \text{ for } p \neq p^*, \tag{5}$$

where we remind the reader that m is the number of possible prices (or the cardinality of set $\mathcal{P}$). $\epsilon(n)$ is a decreasing sequence in [0,1]. Other exploration policies exist, such as the softmax procedure.

We now detail the standard procedure to update the Q-matrix. At iteration n and state s_n, the algorithm samples which price p_n to set using distribution $\theta_n(\cdot)$ and collects its immediate profit $\pi(s_n, p_n)$. The system transitions to state $s_{n+1}(s_n, p_n)$ (denoted by s_{n+1} for simplicity) possibly after rivals' change of prices. The new Q-matrix is updated as:

$$Q_{n+1}(s,p) = \begin{cases} (1-\lambda)Q_n(s,p) + \lambda\left(\pi(s,p) + \delta \max_{p'\in\mathcal{P}} Q_n(s_{n+1}, p')\right) & \text{if } (s,p) = (s_n, p_n) \\ Q_n(s,p) & \text{otherwise} \end{cases} \tag{6}$$

which provides an approximation of the value function at iteration $n+1$. Some inertia is retained via the so-called learning rate $1 - \lambda$: the higher λ, the larger the weight of the recent profits in the updating process. The updating is carried out until convergence of the Q-matrix.

A.3 Decentralized Q-learning

The name "Decentralized Q-learning" has been used for a variety of algorithms. Here, we use it to refer to the algorithm proposed in Arslan and Yüksel (2016). This algorithm is a simple two-timescale implementation of Q-learning. Each player only learns Q-values for their own actions, and thus considers other agents to be part of the environment. These Q-values are learned by initializing two sets of Q-values, one used for defining the behavioral policy of the agent Q_{act} and the other used for learning Q_{val}. Agents learn in batches whose size, K, is one of the hyperparameters of the algorithm. During a batch, all agents keep their policy fixed to generate samples from the (now stationary) environment. These are used to update Q_{val}. At the end of the batch, agents transfer the knowledge they have gained in the form of Q_{val}, to

their behavioral policy with probability $1-\lambda$, i.e.,

$$Q_{\text{act}}^{k+1} = \begin{cases} Q_{\text{val}}^{k} \text{ with probability } 1-\lambda \\ Q_{\text{act}}^{k} \text{ with probability } \lambda. \end{cases}$$

Here, $\lambda \in (0,1)$ is another hyperparameter of the algorithm called "inertia" and k enumerates the batches. Inertia allows the algorithm to break out of cycles and ensures that the algorithm converges to a Nash equilibrium in weakly acyclic games when the batch size is large enough (see Arslan and Yüksel (2016) for a precise statement). In the large batch size limit, the algorithm converges to the best-response strategy adjustment process, which can be analyzed graphically (Meylahn (2023*b*)).

A.4 Deep Q-learning

Deep Q-learning (DQN) follows the logic of Q-learning but uses a neural network to approximate the value function. A possible implementation is as follows: using the notation of previous sections A.1 and A.2, we build a neural network with L layers where the input layer, which processes the input data, corresponds to the state that is observed, and the output layer provides, for each action, the estimation of the value function. Therefore, parameter q coincides with the size of the action set m. Furthermore, the size of the input layer n_1 should be equal to the dimension of states. For instance, if the algorithm observes the past τ prices and there are two players, then $n_1 = 2\tau$. We draw the attention of the reader to the fact that this modeling allows for a representation of states in a continuous set, but still requires actions to be discretized.

The training of DQN can be undertaken as follows. We denote by $\mathbf{w}^n$ the weights of the neural network at iteration n and by $NN(s, \mathbf{w}^n) \in \mathbb{R}^m$ its output for each action when it observes state s as input. We might alternatively write $NN(s, \mathbf{w}^n)(p)$ to specify the output of the neural network for a specific action p (in other words, this is element e of vector $NN(s, \mathbf{w}^n)$ if action p is equal to element p_e of the action set $\{p_1, p_2, \ldots, p_m\}$). At iteration n, a batch with size o of simulations is sampled using distribution $\theta_n(\cdot)$ of actions. This procedure is referred to as experience replay. We then collect states $s^1, s^2, \ldots, s^o$ (and final state s^{o+1}), actions $p^1, p^2, \ldots, p^o$, and immediate profits $\pi^1, \pi^2, \ldots, \pi^o$. We define the output vector y^k corresponding to input state s^k with $1 \leq k \leq o$ as follows:

$$y_e^k = \begin{cases} (1-\lambda) NN(s^k, \mathbf{w}^n)(p^k) + \lambda \left(\pi^k + \delta \max_{p' \in \mathcal{P}} NN(s^{k+1}, \mathbf{w}^n)(p')\right) & \text{for } p_e = p^k \\ NN(s^k, \mathbf{w}^n)(p_{e'}) & \text{otherwise.} \end{cases}$$

In other words, output y^k is an estimation by the NN at iteration n of the long-term discounted profit for all actions when state s^k occurs. The neural network is updated at iterations $n+1$ by finding weights $\mathbf{w}^{n+1}$ that minimize loss function $\mathcal{L}\Big(\mathbf{s}, \mathbf{y}, \mathbf{w}\Big)$ where $\mathbf{s}$ (respectively $\mathbf{y}$) is the concatenation of states $s^1, s^2, \ldots, s^o$ (respectively vectors of long-term discounted profits $y^1, y^2, \ldots, y^o$).

To summarize, DQN is an extension of QL where the Q-matrix is replaced by a neural network, which is trained at each iteration by generating a batch of simulations. Its advantage over QL is that it can capture non-linearities of the value function and can accommodate continuous states. Besides, a splitting between an 'actor' network used for learning the behavior policy and another network used for learning the value function—similar in spirit to what was described for Decentralized Q-Learning in Section A.3—can also be implemented.

A.5 Actor and Critic algorithms

In a critic-only method, the algorithm's objective is to approximate only the value function by employing techniques such as neural networks, without necessarily providing the optimal policy. In contrast, an actor-only method focuses on maximizing the expected long-term discounted payoff by directly optimizing the policy that governs the action choices, without estimating the value function. We call the policy a probability distribution over the set of possible actions $\mathcal{P}$ for a given state s. Following a similar logic as DQN, the algorithm reinforces its learning by iteratively updating its parameters based on new observations of states and rewards, with the goal of maximizing its returns. These updates are performed using specialized gradient techniques, commonly known as policy gradient (PG) methods. Policy gradient methods may, however, suffer from poor numerical performance due to high variance in gradient estimation, leading to longer convergence times and reduced performance. This challenge is amplified in multi-agent settings, where stationarity of the environment is lost, leading to less precise gradient estimates.

Actor-Critic algorithms capitalize on the strengths of both actor-only and critic-only methods. By training an actor and a critic simultaneously, these algorithms offer an integrated solution, leading generally to higher performances. The critic approximates the value function via reinforcement learning principles, using the actor's estimated policy function to generate payoffs. In turn, the actor refines its policy by updating its parameters based on the value function estimated by the critic via policy gradient calculations. Actor-Critic algorithms generally result in better numerical stability with faster convergence compared to either actor-only or critic-only methods.

We now detail a particular instance of this method called Continuous Actor-Critic, inspired by Wang et al. (2016). It treats states and actions as continuous data and computes the actor and critic as neural networks. The input layers of both networks encompass the observed state s. The output layer of the actor network is of size two: it generates two real numbers, η and $\omega \geq 0$, which are used to sample actions via a normal distribution $\mathcal{N}(\eta, \omega)$, where η is the expectation and ω the standard deviation. This distribution is then projected over the action space to generate feasible prices. This procedure defines the policy; of course, other distributions can be considered and the choice of a normal distribution is taken only to facilitate the exposition. The output layer of the critic network represents the long-term expected profit, or value function $V(.)$. In other words, $V(s)$ will estimate the maximum expected long-term discounted profit when state s occurs and the algorithm plays the policy provided by the actor network. Both networks are updated iteratively. At each iteration n, a batch of o experiences is simulated by

sampling actions $p^1, p^2, \ldots, p^o$ from the actor (as estimated at iteration n). Immediate rewards $\pi^1, \pi^2, \ldots, \pi^o$ and states $s^1, s^2, \ldots, s^o$ are observed. For each experience k with $1 \leq k \leq o$, the advantage adv^k is computed as the difference between two terms: the first term is the sum of the immediate reward π^k and the discounted future value estimated by the critic at iteration n: $\delta V^n(s^{k+1})$ (with $V^n(s^{o+1}) = 0$ by convention). The second term is the critic's estimate of the value function at state s^k: $V^n(s^k)$.

$$adv^k = \pi^k + \delta V^n(s^{k+1}) - V^n(s^k), \ \forall 1 \leq k \leq o. \tag{7}$$

The critic is then updated at iteration $n+1$ by minimizing the mean squared advantage across the o experiences, ensuring its predictions of the value function are as accurate as possible. This updating is undertaken by optimizing the weights of the network so that states and values are mapped the most accurately, as described in Section A.1. The actor network is updated by minimizing the following loss function: $-\sum_{k=1}^{o} \ln\left(h^n(p^k)\right) adv^k$, where $h^n(p^k)$ is the probability of action p^k, derived from the normal distribution $\mathcal{N}(\eta^n, \omega^n)$ estimated by the actor network at iteration n. By minimizing this loss, the actor network adjusts parameters η and ω to place greater weight on actions that yielded high advantages. Contrary to Q-learning and DQN, exploration is endogenous as it occurs naturally: during the early stages of learning, the variance ω is set to a high level, allowing for broad exploration of the action space. As learning progresses, ω is reduced (by the updating process of the actor) to focus on prices that produce higher rewards.

A.6 Actor-Critic Q-learning

This algorithm is a particular instance of Actor-Critic methods, where the critic algorithm is modeled via Q-learning. The actor algorithm, for its part, can use a neural network, but it has to provide a discrete policy defined on set $\mathcal{P} = \{p_1, p_2, \ldots, p_m\}$. Therefore, the updating of the critic follows the same procedure as in relationship (6) with the notable difference that a number of simulations are often undertaken between two updating procedures (experience replay).

A.7 Reinforce

The Reinforce algorithm (Williams (1992)) is an actor-based RL method whose objective is to provide the policy that optimizes the long-term discounted profit. In what follows, we assume that the algorithm uses a neural network with weights $\mathbf{w}$ (we will subsequently index them with iteration n when describing the updating procedure). Therefore, we will denote by $\rho(s, \mathbf{w})$ the output policy (or optimal distribution estimated by the neural network) when input state s occurs. Here again, the algorithm is iterative and the updating of $\mathbf{w}$ is as follows: at iteration n, given state s^n and action p^n, we simulate o future actions $p^{n+1}, p^{n+2}, \ldots, p^{n+o}$, and track the resulting states dynamics $s^{n+1}, s^{n+2}, \ldots, s^{n+o}$ along with the corresponding immediate rewards $\pi^{n+1}, \pi^{n+2}, \ldots, \pi^{n+o}$. These simulations are performed under the current policy $\rho(s^n, \mathbf{w}^n)$. The

future discounted profit is then calculated as:

$$G = \sum_{k=1}^{o} \delta^{k-1} \pi^{n+k} \tag{8}$$

and the updating of the network is :

$$\mathbf{w}^{n+1} = \mathbf{w}^{n} + \lambda \; G \; \frac{\partial_{\mathbf{w}} \Big(\rho(s^n, \mathbf{w}^n)(p^n) \Big)}{\rho(s^n, \mathbf{w}^n)(p^n)}. \tag{9}$$

where it is assumed that the policy never assigns zero to any action and where $\partial_x f(.)$ denotes the gradient of function f with respect to variable x. Therefore, when $G > 0$, indicating that the future discounted reward is positive under the current policy, the algorithm's hyperparameters are adjusted following the direction of the policy's gradient, with the aim to increase the likelihood of selecting actions that yield high rewards. The division by probability $\rho(s^n, \mathbf{w}^n)(p^n)$ has a correcting effect, as it amplifies the gradient ascent step when the reward G is positive and the probability of having chosen action p^n was low.

B A summary of the design and results of some articles studying algorithmic collusion

In Tables 1a, 1b, 1c, and 1d we provide a summary of articles on algorithmic collusion in no particular order.

Study	Primary Algorithm	Alternatives	Information Available	Persistent Prices?	Approach to Robustness	Theoretical Method
Waltman and Kaymak (2008)	Q-learning	Self-play		yes	hyperparameters (learning rate, discount rate), # of agents	Numerical analysis, Theory (memoryless case)
Kimbrough and Murphy (2009)	Agent-based modeling	Self-play	stateless	yes (on average)	hyperparameters, objective function, # of agents	Numerical analysis
Calvano et al. (2020*a*,*b*)	Q-learning	Self-play	t-1 prices	yes (on average)	initialization, hyperparameters, # of agents	Numerical analysis
Aouad and den Boer (2021)	Stochastic Approximation	Self-play	stateless	yes (wp 1, when profitable)	theoretical proof	Numerical analysis
Hettich (2021)	DQN	Self-play	t-1 prices	yes (on average)		Numerical analysis
Klein (2021)	Q-learning	Self-play	t-1 price of opponent	yes		
Werner (2021)	Q-learning	Self-play	t-1	yes		Numerical analysis
Asker, Fershtman and Pakes (2022, 2024)	Q-learning (high initialization)	Self-play	t-1 prices	yes (asynchronous algos), no (synchronous algos)		Numerical analysis
Banchio and Mantegazza (2022)	Q-learning, continuous-time approx	Self-play	stateless, t-1 prices	yes (cycles)		Theory, numerical analysis
Cartea, Chang and Penalva (2022)	Deterministic limit of Q-learning, Cross learning and Exp3	Self-play	stateless	yes (with positive probability)	theoretical proof	Theory
Cartea et al. (2022)	Deterministic learning dynamics of broad class of algorithms	Self-play	any finite memory	yes	initial conditions	Theory, numerical analysis
den Boer, Meylahn and Schinkel (2022)	Q-learning	Exp3	t-1 prices			Numerical analysis
Eschenbaum, Mellgren and Zahn (2022)	Q-learning	Self-play	t-1 prices	yes	limiting overfitting during learning and coordinating the environment of training offline	

Table 1a: Summary of Studies on Algorithmic Pricing

Study	Primary Algorithm	Alternatives	Information Available	Persistent Prices?	Approach to Robustness	Theoretical Method
Meylahn and den Boer (2022)	Stochastic Approximation	Best-response function	stateless	yes (wp 1, when profitable)	theoretical proof, performance against alternative class	
Sanchez-Cartas and Katsamakas (2022)	Q-learning, Particle Swarm Optimization (PSO)	Self-play, Q-learning vs PSO, and Q-learning/PSO vs best response	t-1 prices	yes for Q-learning (on average). Much less of PSO (on average)	three market environments (Hotelling, Logit, Linear), Results strongly depend on the market structure	Numerical analysis
Schaefer (2022)	Q-learning	Self-play	t-1	yes	initial conditions, environmental parameters	Numerical analysis
Wang (2022)	Policy gradient methods	Self-play	stateless	no	marginal costs of firms	Numerical analysis
Abada and Lambin (2023)	Q-learning	Self-play	t-1 prices	yes (on average)	entry of new competitors	Numerical analysis
Barfuss and Meylahn (2023)	Decentralized Q-learning	Self-play	t-1 prices	yes	hyperparameters	
Cartea et al. (2023)	State-dependent smooth fictitious play	Self-play	full history (explicit conditioning on t-m prices), transition function, payoffs	yes	theoretical proof	Theory
Conjeaud (2023)	Q-learning, continuous-time approx	Self-play	stateless, t-1 prices	yes for moderate values of exploration rate (cycles), no otherwise		Theory, numerical analysis
Loots and den Boer (2023)	Stochastic Approximation	explore-then-commit policy	stateless	yes (wp 1)	theoretical proof, performance against alternative class	
Meylahn (2023*a*,*b*)	Decentralized Q-learning	Self-play	t-1 prices	yes (with positive probability)	theoretical proof	

Table 1b: Summary of Studies on Algorithmic Pricing

Study	Primary Algorithm	Alternatives	Information Available	Persistent Prices?	Approach to Robustness	Theoretical Method
Possnig (2023)	Actor-Critic Q-learning	Self-play	t-1 actions and their variations	yes	theoretical proof with respect to some hyperparameters	
Wang et al. (2023)	QL and Rule-Based Pricing	QL and Rule-Based Pricing	t-1	yes	Alternative algorithms and market structures	Numerical and Theoretical analysis
Zhang (2023)	Deep QL and Deep Recurrent QL	Self-play	t-1	no	no	Numerical analysis
Abada, Lambin and Tchakarov (2024)	Q-learning	Exp3, REINFORCE, AC	t-1 prices	yes (Q-learning under self-play)		Theory and numerical analysis
Arunachaleswaran et al. (2024)	Repeated Stackelberg between No-regret and static Best response	Self-play	full history	yes (wp 1)	Bertrand, logit environments, self play (convergence to static Nash)	Theory
Cho and Williams (2024)	Model-based reaction functions	Self-play	full history	yes (on average, collusive episodes)	stochastic demand	Theory, numerical analysis
Deng, Schiffer and Bichler (2024)	Q-learning, DQL, SAC, PPO	Self-play	t-1 prices	yes on average, but only with specific environments-algorithm pairs		Numerical analysis
Dolgopolov (2024)	Q-learning (alpha converges to 0)	Self-play	no past prices (prisoner dilemma)	no if epsilon-greedy, yes if boltzman exploration		Theory, numerical analysis
Epivent and Lambin (2024)	Q-learning	Self-play	t-1 prices	yes	shifting action space, allowing deviations to high prices	
Fish, Gonczarowski and Shorrer (2024)	LLM	Q-learning	t-100 prices	yes	different prompts, asymmetric algorithms, Performance against Q-learning	Numerical analysis

Table 1c: Summary of Studies on Algorithmic Pricing

Study	**Primary Algorithm**	**Alternatives**	**Information Available**	**Persistent Prices?**	**Approach to Robustness**	**Theoretical Method**
Lambin (2024)	Q-learning	Self-play	stateless, t-1 prices, full history	yes (wp 1)	not discussed	Theory, numerical analysis
Macrì and Lillo (2024)	Double Deep Q-learning	Self-play	t-1 market price	yes (on average)	variations in volatility of the environment	Numerical analysis
Sadoune, Joanis and Lodi (2024)	Multi-armed bandits, Double Deep Q-learning, Actor-critic, UCB	Self-play	stateless bandits, past action frequencies (only two possible actions: competition, collusion)	no (on average) except in the "most naive settings"	variations in market shares, number of agents	Numerical analysis
Schlechtinger (2024)	DQN, PPO (Proximal Policy Optimization)	Self-play	t-1 prices	yes (on average)	hyperparameters and 'ablation' (limiting the gradient algorithm when learning)	Numerical and some theory on a toy model with three actions
Xu and Zhao (2024)	Q-learning	Self-play	stateless	yes	the rate of decrease of exploration	Theory and numerical simulations

Table 1d: Summary of Studies on Algorithmic Pricing